ทักษะปัญญาประดิษฐ์

เพื่อการจำลองฟิสิกส์ 3 มิติ ห้องปฏิบัติการเสมือนจริง และ AIoT

Series of AI SKILL — Volume 01: Virtual Labs, 3D Physics Simulation & AIoT

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

ศูนย์ความเป็นเลิศด้านสิ่งแวดล้อมและพลังงาน (LEQs) • พ.ศ. 2569

Contents

1 สถาปัตยกรรมการจำลองฟิสิกส์ 3 มิติด้วย Three.js และ PBR Shaders 1
1.1 รากฐานระบบเรนเดอร์ WebGL และการบริหารจัดการหน่วยความจำ 1
1.2 การสร้างพื้นผิว Physically Based Rendering และแสงเงา 2
1.3 การจำลองฟิสิกส์เพนดูลัมฮาร์มอนิกและการลดรูปวงจร 6 เหลี่ยม 2

2 ระบบตรวจจับและสั่งการไร้สัมผัสด้วย AR MediaPipe Hands 4
2.1 โครงสร้างเรขาคณิตของมือมนุษย์ 21 จุดข้อต่อ . . . 4
2.2 อัลกอริทึมการคำนวณเวกเตอร์ท่าทางมือ 5
2.3 การผสานระบบเสียงอคูสติกสดผ่าน Web Audio API 5

3 ระบบ AIoT และการประมวลผลปัญญาประดิษฐ์ฝังตัวบนขอบ 6
3.1 การสื่อสารฮาร์ดแวร์เซนเซอร์วัดค่าดินผ่าน RS485 Modbus RTU 6
3.2 การฝึกสอนโมเดล Ordinary Least Squares บน Edge Device 7

A คลังรหัสต้นฉบับและองค์ประกอบสกิลจำลองฟิสิกส์ 3 มิติ และ AIoT 8
A.1 สกิล: nextgen-physics-virtual-lab-architect . . . 8
A.1.1 โค้ดแกนกลาง: เอนจินคำนวณเศษส่วนแท้ Euclidean GCD 8
A.1.2 โค้ดแกนกลาง: การคำนวณความต้านทานสมมูลวงจรหกเหลี่ยม 9

A.2 สกิล: ar-simulation-builder 10
A.2.1 โค้ดแกนกลาง: ระบบตรวจจับการจีบนิ้วและการสังเคราะห์เสียง 10
A.3 สกิล: soil-pht-aiot-architect 11
A.3.1 โค้ดแกนกลาง: การคำนวณผลรวมตรวจสอบ Modbus CRC16 12

บทที่ 1

สถาปัตยกรรมการจำลองฟิสิกส์ 3 มิติด้วย Three.js และ PBR Shaders

การพัฒนาระบบห้องปฏิบัติการเสมือนจริงในปัจจุบัน จำเป็นต้องอาศัยการผสมผสานระหว่างฟิสิกส์เชิงทฤษฎีและคอมพิวเตอร์กราฟิกส์สมรรถนะสูง เพื่อให้ผู้เรียนเกิดประสบการณ์เชิงลึกเสมือนได้สัมผัสอุปกรณ์จริงในห้องปฏิบัติการ

1.1 รากฐานระบบเรนเดอร์ WebGL และการบริหารจัดการหน่วยความจำ

การทำงานของ WebGL บนเว็บเบราว์เซอร์สมัยใหม่ เป็นการสื่อสารโดยตรงกับหน่วยประมวลผลกราฟิกส์ (GPU) ผ่าน Vertex Shaders และ Fragment Shaders โดยมีสถาปัตยกรรมหลักดังแสดงในกรอบแนวคิด

[ทฤษฎีและสมการ] หลักการสำคัญ การลดทอนการจัดสรรหน่วยความจำ (Zero Garbage Collection)

ในแอนิเมชันความเร็ว 60 เฟรมต่อวินาที การสร้างวัตถุใหม่ในลูปเรนเดอร์ เช่น `new THREE.Vector3()` ส่งผลให้ Garbage

Collector ทำงานบ่อยครั้ง เกิดอาการกระตุก (Frame Drop) แนวทางที่ถูกต้องคือการใช้ **Object Reuse** และคำนวณผ่านตัวแปรระดับโกลบอลที่จัดสรรไว้ล่วงหน้า

1.2 การสร้างพื้นผิว Physically Based Rendering และแสงเงา

การสร้างแบบจำลองอุปกรณ์วิทยาศาสตร์ เช่น ตัวต้านทาน ขดลวด หรือลูกตุ้มโลหะ ต้องใช้ `MeshStandardMaterial` หรือ `MeshPhysicalMaterial` ซึ่งจำลองการสะท้อนของแสงตามหลักทัศนศาสตร์จริง:

$$f_r(\omega_i, \omega_o) = \frac{D(h)F(\omega_i, h)G(\omega_i, \omega_o, h)}{4(\omega_i \cdot n)(\omega_o \cdot n)} \tag{1.1}$$

โดยที่ $D(h)$ คือการกระจายของพื้นผิวจุลภาค (Microfacet Distribution), F คือพจน์เฟรสเนล (Fresnel Reflectance), และ G คือสัมประสิทธิ์การบดบังของพื้นผิว (Geometric Attenuation)

1.3 การจำลองฟิสิกส์เพนดูลัมฮาร์มอนิกและการลดรูปวงจร 6 เหลี่ยม

ในระบบจำลองการเคลื่อนที่ของลูกตุ้มเพนดูลัมแบบง่าย คาบเวลาการแกว่งที่มุมขนาดเล็ก ($\theta < 10^\circ$) เป็นไปตามสมการ:

$$T = 2\pi\sqrt{\frac{L}{g}} \tag{1.2}$$

ส่วนในวงจรไฟฟ้าแบบโครงข่ายหกเหลี่ยมด้านเท่า (Hexagonal Resistor Network) ที่มีตัวต้านทานขนาด R เท่ากันทั้ง 6 ด้าน เมื่อต่อขั้วขนานระหว่างจุดยอดตรงข้าม ความต้านทานสมมูลรวม R_{eq} สามารถคำนวณได้จากหลักการสมมาตรของศักย์ไฟฟ้า:

$$R_{eq} = \frac{5}{6}R \tag{1.3}$$

เมื่อ $R = 1.0\,\Omega$ จะได้ค่าความต้านทานสมมูลเท่ากับ $0.8333\,\Omega$ ซึ่งตรงกับผลลัพธ์ที่วัดได้จริงจากแอมมิเตอร์และโวลต์มิเตอร์ในแบบจำลอง 3 มิติ

บทที่ 2

ระบบตรวจจับและสั่งการไร้สัมผัสด้วย AR MediaPipe Hands

การควบคุมการทดลองวิทยาศาสตร์เสมือนจริงโดยไม่ใช้เมาส์หรือคีย์บอร์ด ช่วยเสริมสร้างทักษะการลงมือปฏิบัติจริงเชิงประจักษ์ (Kinesthetic Learning) ผ่านการวิเคราะห์โครงข่ายประสาทเทียมของมือมนุษย์แบบเรียลไทม์

2.1 โครงสร้างเรขาคณิตของมือมนุษย์ 21 จุดข้อต่อ

โมเดลคอมพิวเตอร์วิทัศน์ MediaPipe Hands สกัดจุดพิกัดสามมิติ (x, y, z) ของข้อนิ้วจำนวน 21 จุด โดยมีจุดอ้างอิงสำคัญ ได้แก่:

- **WRIST (จุดที่ 0):** ข้อมือส่วนล่างสำหรับกำหนดเวกเตอร์การเคลื่อนที่ของมือ
- **THUMB_TIP (จุดที่ 4):** ปลายนิ้วหัวแม่มือ
- **INDEX_FINGER_TIP (จุดที่ 8):** ปลายนิ้วชี้สำหรับใช้เป็นเคอร์เซอร์ระบุตำแหน่ง

- **MIDDLE_FINGER_TIP (จุดที่ 12):** ปลายนิ้วกลางสำหรับประเมินการกำมือ

2.2 อัลกอริทึมการคำนวณเวกเตอร์ท่าทางมือ

การจำแนกท่าทาง (Gesture Classification) ใช้การคำนวณระยะทางแบบยูคลิด (Euclidean Distance) ระหว่างปลายนิ้วชี้และปลายนิ้วหัวแม่มือ:

$$d_{\text{pinch}} = \sqrt{(x_8 - x_4)^2 + (y_8 - y_4)^2 + (z_8 - z_4)^2} \tag{2.1}$$

เมื่อ $d_{\text{pinch}} < \delta_{\text{threshold}}$ (กำหนดค่าที่ 0.05 ในพิกัดนอร์แมลไลซ์) ระบบจะส่งสัญญาณสถานะ `PINCH_ACTIVE` เพื่อจำลองการหยิบจับสายไฟ ขยับลูกตุ้ม หรือหมุนปุ่มปรับค่าบนแผงควบคุม

2.3 การผสานระบบเสียงอคูสติกสดผ่าน Web Audio API

เมื่อมือของผู้ใช้สัมผัสหรือปล่อยอุปกรณ์ ระบบจะสังเคราะห์คลื่นเสียงความถี่เฉพาะผ่าน Web Audio API โดยไม่ต้องโหลดไฟล์เสียงภายนอก:

$$f_{\text{click}} = 880\,\text{Hz}, \quad \tau_{\text{decay}} = 0.05\,\text{s} \tag{2.2}$$

ทำให้ผู้เรียนได้รับข้อมูลป้อนกลับแบบสองทาง (Haptic Audio Feedback) ที่ตอบสนองในระดับมิลลิวินาที

บทที่ 3

ระบบ AIoT และการประมวลผลปัญญาประดิษฐ์ฝังตัวบนขอบ

การเชื่อมต่อโลกเสมือนจริงเข้ากับเซนเซอร์ภาคสนาม เป็นแกนกลางของการเกษตรแม่นยำ (Precision Agriculture) และฟิสิกส์ประยุกต์ โดยใช้โมเดลปัญญาประดิษฐ์ประมวลผล ณ จุดกำเนิดข้อมูล (Edge AI)

3.1 การสื่อสารฮาร์ดแวร์เซนเซอร์วัดค่าดินผ่าน RS485 Modbus RTU

หัววัดคุณสมบัติดินแบบแท่งโลหะหลายพารามิเตอร์ (Multi-parameter Soil Probe) ส่งข้อมูลผ่านโปรโตคอลอุตสาหกรรม Modbus RTU ด้วยความเร็ว 9600 bps โดยมีโครงสร้างเฟรมคำสั่งมาตรฐาน (Request Frame) ดังนี้:

- **Slave ID (1 Byte):** รหัสประจำตัวอุปกรณ์ เช่น `0x01`
- **Function Code (1 Byte):** ฟังก์ชันอ่านค่ารีจิสเตอร์ `0x03`
- **Starting Address (2 Bytes):** ตำแหน่งแอดเดรสเริ่มต้น เช่น `0x0000`
- **Quantity of Registers (2 Bytes):** จำนวนรีจิสเตอร์ที่ต้องการอ่าน

- **CRC16 Checksum (2 Bytes):** รหัสตรวจสอบความถูกต้องของข้อมูล

3.2 การฝึกสอนโมเดล Ordinary Least Squares บน Edge Device

ในการปรับเทียบความถูกต้องของค่าความเป็นกรด-ด่าง (pH) และความชื้นในดิน อัลกอริทึมการถดถอยเชิงเส้นพหุคูณคำนวณผ่านเวกเตอร์เมทริกซ์ในหน่วยความจำของบอร์ดประมวลผล:

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{3.1}$$

โดยที่ **X** คือเมทริกซ์ของสัญญาณไฟฟ้าและอุณหภูมิ และ **y** คือค่ามาตรฐานที่วัดได้จากห้องปฏิบัติการเคมีมาตรฐาน ผลลัพธ์ที่ได้มีความแม่นยำสูงและตอบสนองได้แบบเรียลไทม์โดยไม่ต้องพึ่งพาเซิร์ฟเวอร์คลาวด์ภายนอก

บทที่ A

คลังรหัสต้นฉบับและองค์ประกอบสกิลจำลองฟิสิกส์ 3 มิติ และ AIoT

ภาคผนวกนี้รวบรวมข้อมูลจำเพาะ โครงสร้างเมทาดาทา และโค้ดอัลกอริทึมของสกิลปัญญาประดิษฐ์ (Agent Skills) ประจำเล่มที่ 1 อย่างละเอียด เพื่อเป็นคู่มืออ้างอิงเชิงวิศวกรรมซอฟต์แวร์

A.1 สกิล: nextgen-physics-virtual-lab-architect

[แผนการสอน] ข้อมูลจำเพาะและเมทาดาทาของสกิล

ชื่อสกิล: `nextgen-physics-virtual-lab-architect`
วัตถุประสงค์: ออกแบบและสร้างห้องปฏิบัติการฟิสิกส์ 3 มิติเสมือนจริงตามมาตรฐานโอลิมปิกวิชาการ (IJSO Standard)
ขอบเขตเทคโนโลยี: Three.js PBR Shaders, Raycasting, SVG Vector Engine, Euclidean GCD

A.1.1 โค้ดแกนกลาง: เอนจินคำนวณเศษส่วนแท้ Euclidean GCD

```javascript
function gcd(a, b) {
  a = Math.abs(Math.round(a));
  b = Math.abs(Math.round(b));
  while (b > 0) {
    const t = b;
    b = a % b;
    a = t;
  }
  return a;
}

function simplifyFraction(numerator, denominator) {
  if (denominator === 0) return { num: 0, den: 1 };
  const common = gcd(numerator, denominator);
  return {
    num: numerator / common,
    den: denominator / common,
    str: `${numerator / common}/${denominator / common}`
  };
}
```

Listing A.1: Euclidean GCD Engine สำหรับรักษาความแม่นยำทางฟิสิกส์

A.1.2 โค้ดแกนกลาง: การคำนวณความต้านทานสมมูลวงจรหกเหลี่ยม

```javascript
function calculateHexEquivalentResistance(nodeP, nodeQ, resistors) {
  // nodeP, nodeQ: Indices 0 to 5 (Nodes a, b, c, d, e, f)
  const totalR = resistors.reduce((sum, val) => sum + val, 0);

  // Calculate direct path resistance (r1)
  let r1 = 0;
  let curr = nodeP;
  while (curr !== nodeQ) {
    r1 += resistors[curr];
    curr = (curr + 1) % 6;
  }

  // Calculate indirect path resistance (r2)
  const r2 = totalR - r1;

  // Parallel equivalence: Req = (r1 * r2) / (r1 + r2)
```

```
    const req = (r1 * r2) / totalR;
    const fraction = simplifyFraction(r1 * r2, totalR);

    return { r1, r2, req, fraction };
}
```

Listing A.2: Hexagonal Circuit Reduction Algorithm

A.2 สกิล: ar-simulation-builder

[แผนการสอน] ข้อมูลจำเพาะและเมทาดาทาของสกิล

ชื่อสกิล: `ar-simulation-builder`
วัตถุประสงค์: สร้างระบบโต้ตอบ 3 มิติไร้สัมผัสผ่านการตรวจจับท่าทางมือด้วย MediaPipe Hands 60 FPS
ขอบเขตเทคโนโลยี: MediaPipe Vision, Web Audio API Acoustic Synthesizer, Zero-GC Vector Math

A.2.1 โค้ดแกนกลาง: ระบบตรวจจับการจีบนิ้วและการสังเคราะห์เสียง

```
class TouchlessHandController {
  constructor(audioCtx) {
    this.audioCtx = audioCtx;
    this.isPinching = false;
    this.threshold = 0.05; // Normalized screen space
        distance
  }

  detectPinch(landmarks) {
    const thumb = landmarks[4];
    const index = landmarks[8];
    const dist = Math.hypot(
      thumb.x - index.x,
      thumb.y - index.y,
      thumb.z - index.z
    );

    if (dist < this.threshold && !this.isPinching) {
```

```
        this.isPinching = true;
        this.playHapticSound(880, 0.05); // High-pitch click
        return 'PINCH_START';
      } else if (dist >= this.threshold && this.isPinching) {
        this.isPinching = false;
        this.playHapticSound(440, 0.04); // Release click
        return 'PINCH_END';
      }
      return this.isPinching ? 'PINCHING' : 'IDLE';
    }

    playHapticSound(freq, duration) {
      const osc = this.audioCtx.createOscillator();
      const gain = this.audioCtx.createGain();
      osc.frequency.value = freq;
      gain.gain.setValueAtTime(0.15, this.audioCtx.currentTime
          );
      gain.gain.exponentialRampToValueAtTime(0.001, this.
          audioCtx.currentTime + duration);
      osc.connect(gain);
      gain.connect(this.audioCtx.destination);
      osc.start();
      osc.stop(this.audioCtx.currentTime + duration);
    }
  }
```

Listing A.3: Pinch Gesture Detection and Web Audio Feedback

A.3 สกิล: soil-pht-aiot-architect

[แผนการสอน] ข้อมูลจำเพาะและเมทาดาทาของสกิล

ชื่อสกิล: `soil-pht-aiot-architect`
วัตถุประสงค์: ควบคุมการรับส่งข้อมูลผ่านฮาร์ดแวร์ RS485 Modbus RTU และฝึกสอนโมเดล Edge AI OLS
ขอบเขตเทคโนโลยี: Dart/Flutter Serial Port, Modbus CRC16, Ordinary Least Squares

A.3.1 โค้ดแกนกลาง: การคำนวณผลรวมตรวจสอบ Modbus CRC16

```
function calculateCRC16(buffer) {
  let crc = 0xFFFF;
  for (let pos = 0; pos < buffer.length; pos++) {
    crc ^= buffer[pos];
    for (let i = 8; i !== 0; i--) {
      if ((crc & 0x0001) !== 0) {
        crc >>= 1;
        crc ^= 0xA001;
      } else {
        crc >>= 1;
      }
    }
  }
  return crc; // 16-bit CRC Checksum
}
```

Listing A.4: Modbus CRC16 Verification Function